Software Engineering

LAB 7 – FUZZ TESTING USING HYPOTHESIS

Name: Thrupthi G D
SRN: PES1UG24AM476
Experiment: Fuzz Testing for Input Validation Bugs

1. Objective

To explore fuzz testing as a software testing technique by identifying and fixing input validation bugs using the Hypothesis property-based testing library.

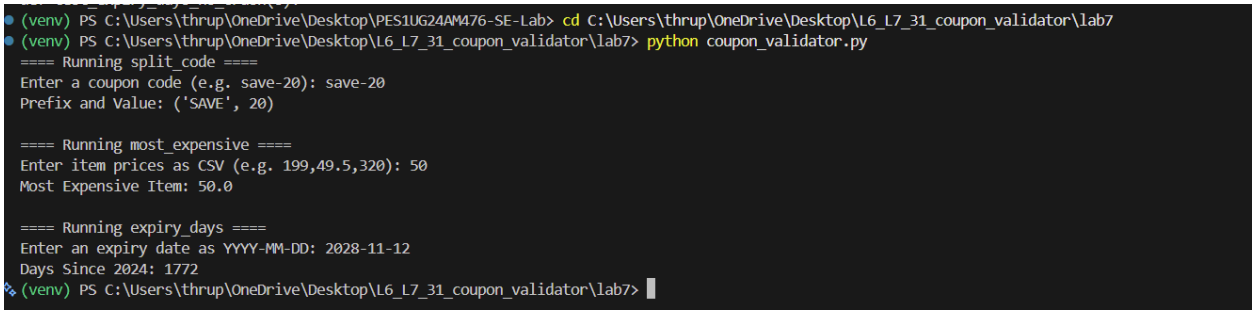
2. Application Overview

The Coupon Validator application contains three functions:

- `split_code()` – separates a coupon prefix and numeric value.
- `most_expensive()` – finds the highest price from comma-separated values.
- `expiry_days()` – calculates a day count relative to 2024.

3. Original Application Execution

The original application was executed with valid inputs to verify its basic functionality.



4. Hypothesis Fuzz Testing

Hypothesis was used to generate unexpected and malformed inputs automatically.

Three property-based tests were implemented to test all three application functions.

```
Problems 4  Output  Debug Console  Ports  Terminal

(venv) PS C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7> python -m pip show hypothesis
>> python -m pytest --version
Name: hypothesis
Version: 6.130.15
Summary: The property-based testing library for Python
Home-page: https://hypothesis.works
Author:
Author-email: "David R. MacIver and Zac Hatfield-Dodds" <david@drmaciver.com>
License-Expression: MPL-2.0
Location: C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\venv\Lib\site-packages
Requires: sortedcontainers
Required-by:
pytest 9.2.0
(venv) PS C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7> python -m pytest test_coupon_validator.py -v
============================= test session starts ==============================
platform win32 -- Python 3.15.0, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\venv\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7
plugins: hypothesis-6.130.15
collected 3 items

test_coupon_validator.py::test_split_code_no_crash FAILED                [ 33%]
test_coupon_validator.py::test_most_expensive_no_crash FAILED            [ 66%]
test_coupon_validator.py::test_expiry_days_no_crash FAILED               [100%]

=================================== FAILURES ===================================
_____________________________ test_split_code_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 16, in test_split_code_no_crash
  |     def test_split_code_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 17, in test_split_code_no_crash
    |     result = split_code(s)
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\coupon_validator.py", line 3, in split_code
    |     return prefix, int(value)
    |                    ^^^^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    | Falling test case: test_split_code_no_crash(
    |     s="-",
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 17, in test_split_code_no_crash
    |     result = split_code(s)
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\coupon_validator.py", line 2, in split_code
    |     prefix, value = code.strip().upper().split("-")
    |     ^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    | Falling test case: test_split_code_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 17, in test_split_code_no_crash
    |     result = split_code(s)
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\coupon_validator.py", line 2, in split_code
    |     prefix, value = code.strip().upper().split("-")
    |                     ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    | Falling test case: test_split_code_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____________________________ test_most_expensive_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 38, in test_most_expensive_no_crash
  |     def test_most_expensive_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 39, in test_most_expensive_no_crash
    |     result = most_expensive(s)
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\coupon_validator.py", line 6, in most_expensive
    |     prices = [float(p) for p in prices_csv.split(",")]
    |               ~~~~~^^^
    | ValueError: could not convert string to float: ''
    | Falling test case: test_most_expensive_no_crash(
    |     s='',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 39, in test_most_expensive_no_crash
    |     result = most_expensive(s)
    |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\coupon_validator.py", line 6, in most_expensive
    |     prices = [float(p) for p in prices_csv.split(",")]
    |                             ^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    | Falling test case: test_most_expensive_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____________________________ test_expiry_days_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\thrup\OneDrive\Desktop\L6_L7_3I_coupon_validator\lab7\test_coupon_validator.py", line 62, in test_expiry_days_no_crash
  |     def test_expiry_days_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
```

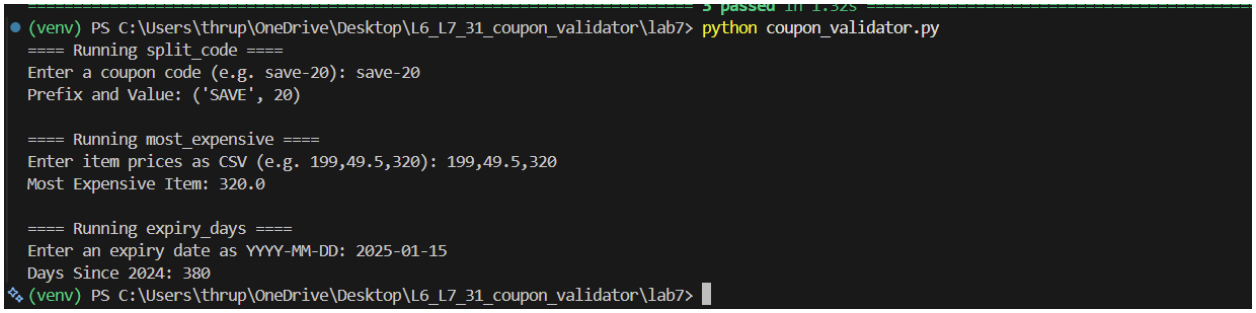
Bugs Identified

- Invalid coupon formats caused ValueError and unpacking errors.
- Non-string coupon inputs caused AttributeError.
- Empty or invalid price lists caused ValueError.
- Non-string price inputs caused AttributeError.
- Invalid date inputs caused ValueError and AttributeError.

5. Bug Fixes and Input Validation

The application was updated with defensive input validation.

- Type checking was added.
- Coupon formatting was validated.
- Price values were checked for validity.
- Date parsing and calendar validation were implemented.
- Invalid inputs return `None` instead of causing exceptions.

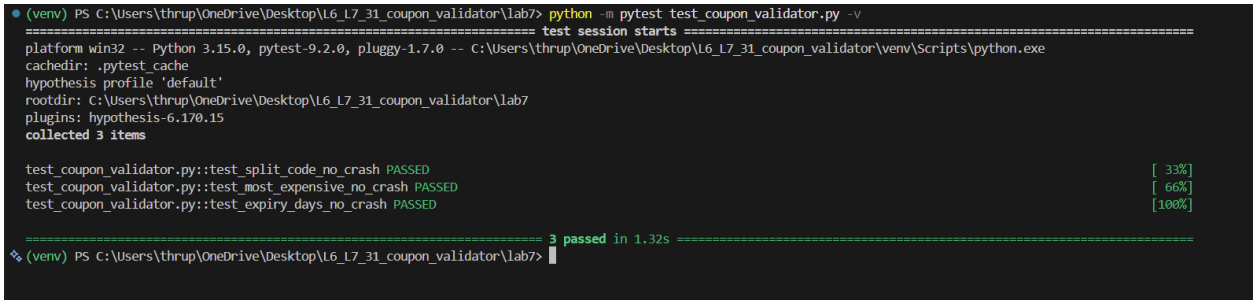


6. Final Fuzz Testing Results

The updated application was tested again using Hypothesis.

All three property-based test functions passed successfully.

Test Function	Result
test_split_code_no_crash	PASSED
test_most_expensive_no_crash	PASSED
test_expiry_days_no_crash	PASSED



7. Conclusion

Fuzz testing using Hypothesis successfully identified input validation bugs in the Coupon Validator application. Defensive programming techniques were implemented to handle unexpected and malformed inputs. After fixing the application, all three property-based tests passed, demonstrating improved input robustness.